

CPU Scheduling Algorithms Lab

Dr. Ahmed Awais

April 18, 2025

CPU Scheduling Algorithms

For attempting this Lab Manual, I recommend recall algorithmic logic (algorithm) of each Scheduling Criterion, and then begin with writing Python code.

Problem Statement

In this assignment, you will implement various **CPU scheduling algorithms** to manage process execution in an operating system. You will create a **Process** class to represent each process, define relevant time variants, and implement scheduling techniques.

Objectives

- Implement 8 CPU scheduling algorithms (currently only two FCFS and SJF non-preemptive both with BT and AT assumed as known)
- Compute performance metrics (WT, TAT)

Process Class Definition

```
class Process:
    def __init__(self, pid, arrival_time, burst_time, priority=None, queue_level=None):
        self.pid = pid
        self.arrival_time = arrival_time
        self.burst_time = burst_time
        self.remaining_time = burst_time
        self.waiting_time = 0
        self.turnaround_time = 0
        self.priority = priority
        self.queue_level = queue_level
```

Scheduling Algorithms

1. First-Come, First-Served (FCFS)

```
def FCFS(processes):
    Sort processes by arrival_time
    current_time = 0
```

```

for process in processes:
    if process.arrival_time > current_time:
        current_time = process.arrival_time
    process.waiting_time = current_time - process.arrival_time
    current_time += process.burst_time
    process.turnaround_time = current_time - process.
        arrival_time
return processes

```

2. Shortest Job First (SJF)

```

def SJF(processes):
    Sort processes by arrival_time
    current_time = 0
    completed = 0
    while completed < len(processes):
        available = [p for p in processes if p.arrival_time <=
            current_time and p.remaining_time > 0]
        if not available:
            current_time += 1
            continue
        shortest = min(available, key=lambda x: x.burst_time)
        process.waiting_time = current_time - process.arrival_time
        current_time += process.burst_time
        process.turnaround_time = current_time - process.
            arrival_time
        completed += 1
    return processes

```

Performance Metrics

Compute average waiting and turnaround times:
Write your logic.

Gantt Chart Visualization

If you want to add Visualization using any library it's totally up to you (Optional) - Otherwise simple printing would be enough with each iteration.

Submission Guidelines

- Python script (your_ID_scheduling_algorithms.py/txt/ipynb etc)
- Well-commented code
- Sample outputs for 10 different process sets (a few from direct inputs + a few file reading for a minimum of 3 Process to 10 Processes with different algorithms) such as
- Now, solve above your selected inputs on A4 Paper, I will check them during Lab.
- Deadline: **April 23, 2025**